Machine Learning for Finance

Supervised Learning - Classification

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Introduction
2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART – Classification and Regression Trees
4. Evaluating Classification Models

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
4. Evaluating Classification Models

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)
6. Evaluate out-of-sample performance

Q: Where do the "labels" (long/short) come from?
A: Labels are imposed ex-post.

By contrast, in Classification, we're going to handle the labels differently.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

- We define an outcome Y (*success* / regime / event) *a priori*.
- We model $P(Y|X)$ or a decision rule $g(X)$ using labeled data (X, Y).
- We then predict Y (or $P(Y = 1|X)$) for new observations.

Key difference: classification trains on *fixed labeled outcomes*; clustering does not.

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then learn the $P(Y = 1|X)$
- Q: How frame as a prediction problem? How frame as a forecasting problem?
- A: Prediction $\rightarrow P(Y_t|X_t)$ Forecast $\rightarrow P(Y_{t=1+1}|X)$

Cautionary Note About Regime Dependence

Regimes are inherently time-series objects (persistence, serial dependence).

Today: we treat observations (X_t, Y_t) as if they are i.i.d. for the purpose of learning the *mapping* from X to Y.

- We focus on *classification methods* and their objectives (loss/risk).
- We will *not* model state persistence or transition dynamics here.

Later: we revisit dependence (blocked / rolling CV, sequential decision costs, regime persistence, etc.).

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

- fitted values can violate the support of Y (e.g., probabilities < 0 or > 1),
- errors are heteroskedastic by construction,
- decision thresholds / costs become central.

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).
- **Corner solutions:** portfolio weight at 0 for many assets; participation decisions.

We will begin with binary, then move to multinomial, but likely won't cover Ordered or Corner Solutions.

Outline

1. Introduction
2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
3. Multi Class Classification
4. Evaluating Classification Models

Outline

1. Introduction
2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
3. Multi Class Classification
4. Evaluating Classification Models

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{otherwise} \end{cases} \qquad X_t \in \mathbb{R}^K \text{ (features)}$$

As mentioned earlier, we can treat this as prediction or forecasting.
Linear Probability Model:

$$P(Y_t = 1|X_t) = \mathbb{E}[Y_t|X_t] = \beta_0 + \beta^\top X_t$$

Estimate by OLS:

$$(\hat{\beta}_0, \hat{\beta}) = \arg\min_{\beta_0,\beta} \sum_{t=1}^{T} (Y_t - \beta_0 - \beta^\top X_t)^2$$

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

Interpretation:

- $\hat{p}_t$ is a *probability forecast* (not a regime label).
- To produce a regime prediction/forecast, choose a threshold τ:

$$\hat{Y}_t(\tau) = \mathbf{1}\{\hat{p}_t \geq \tau\}$$

- e.g. If probability of Bear $> 60\%$ we classify as Bear market.

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

$$\mathrm{Var}(Y_t|X_t) = p_t(1 - p_t)$$

   So OLS standard errors need robust corrections.
3. **Rare events:** if bear is rare, accuracy can be misleading
4. **Time dependence (caveat):** regimes are persistent; naive i.i.d. inference is not appropriate for time-series standard errors.

Conclusion: LPM is a useful baseline, but we want a model with $0 < p < 1$ by construction.

Outline

1. Introduction
2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
3. Multi Class Classification
4. Evaluating Classification Models

What is the Probability of a Bear Market? (Logit/Probit)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

The Linear Probability Model assumes

$$P(Y_t = 1 \mid x_t) = \beta_0 + \beta^\top x_t$$

Problem

The linear score

$$\beta_0 + \beta^\top x_t \in (-\infty, \infty)$$

so predicted probabilities can be invalid (e.g. $p_t < 0$ or $p_t > 1$).

Idea

Transform the linear score into a valid probability.

Logit and Probit

Define the linear score

$$\eta_t = \beta_0 + \beta^\top x_t$$

Map the score into a probability via link function G:

$$P(Y_t = 1 \mid x_t) = G(\eta_t)$$

where

$$G : \mathbb{R} \to (0, 1)$$

Two common choices for the link function

- Logit

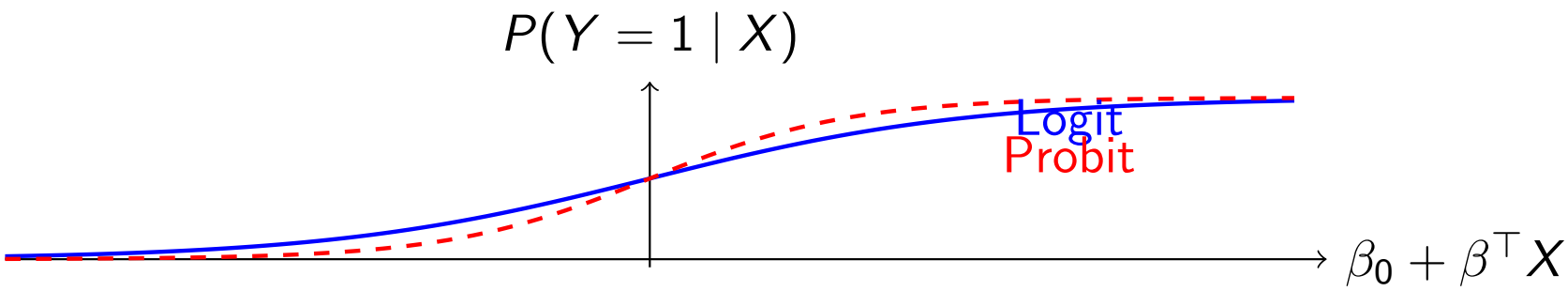
$$G(\eta) = \frac{1}{1 + e^{-\eta}}$$

- Probit

$$G(\eta) = \Phi(\eta)$$

Logit vs Probit

Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$



- Both models transform a linear score into a probability.
- The two curves are nearly identical.
- In practice, logit and probit typically produce very similar predicted probabilities.

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.
Assume conditional Bernoulli:

$$\mathbb{P}(Y_t \mid x_t; \beta) = p_t(\beta)^{Y_t} \left[1 - p_t(\beta)\right]^{1-Y_t}$$

Log-likelihood:

$$\ell(\beta) = \sum_{t=1}^{T} \Big(Y_t \log p_t(\beta) + (1 - Y_t) \log(1 - p_t(\beta)) \Big)$$

$$\hat{\beta} = \arg\max_{\beta} \ell(\beta)$$

Estimated numerically.

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Example (Bear market case study)

- Suppose $\beta_0 + \beta^\top x_t = -1.39$
- Logit: $p_t \approx 0.20$; i.e. 20% probability of a Bear
- Probit: $p_t \approx 0.18$; i.e. 18% probability of a Bear

The model outputs a probability of Bear conditional on the observed features x_t.

Converting Probability to Model Scale

Suppose that using the observed features x_t the model predicts a baseline probability of Bear $p_t = 0.20$

Logit

Convert probability to odds:

$$\text{odds} = \frac{p}{1-p} = \frac{0.20}{0.80} = 0.25$$

Probit

Convert probability to the latent score:

$$\eta_t = \Phi^{-1}(0.20) \approx -0.84$$

These are simply different ways of representing the same probability.

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84
After $\Delta CS = +1$	Odds $\times e^{0.8}$	Score +0.8
New value	0.56	-0.04
Converted to probability	0.36	0.48

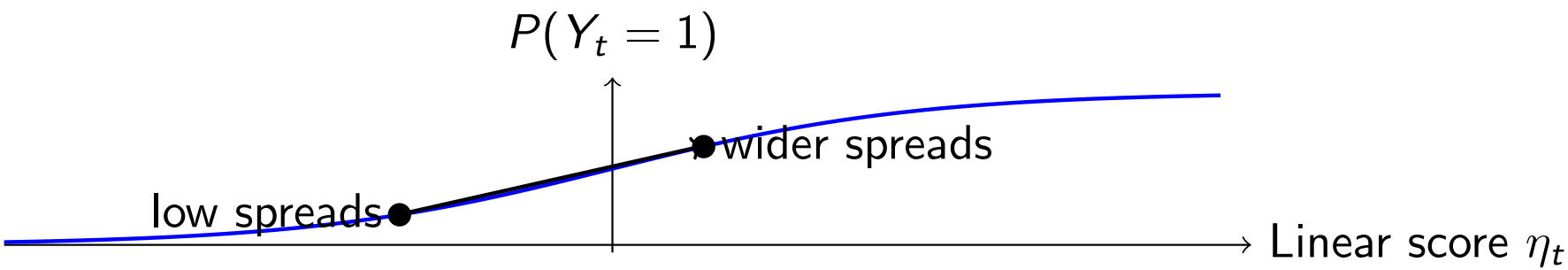
An increase in credit spreads substantially raises the probability of a Bear regime in both models.

How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

An increase in a feature (e.g., credit spreads) changes η_t, which moves the predicted probability along the S-curve.



- Features affect the probability of Bear by shifting the linear score η_t.
- Because the curve is nonlinear, the same change in η_t can produce different probability changes depending on where you start.

Summary: LPM vs Logit/Probit

	LPM	**Logit/Probit**
Range of $\hat{p}$	Unbounded	$(0, 1)$
Estimation	OLS	MLE
Interpretation	Linear in X	Nonlinear in X
Errors	Heteroskedastic	Bernoulli variance
Best use	Baseline	Standard probabilistic classifier

Outline

1. Introduction
2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
3. Multi Class Classification
4. Evaluating Classification Models

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

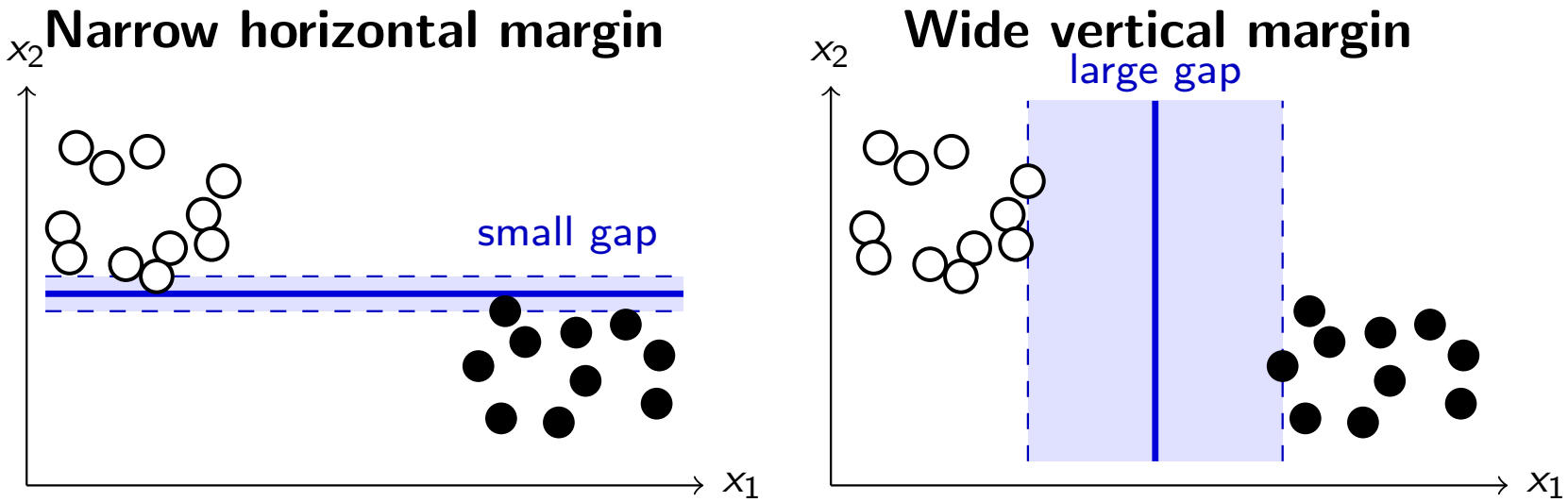
Goal: learn a rule that maps features x_t to a regime Y_t:

$$\widehat{Y}_t = \text{sign}\left(w^\top x_t + b\right)$$

- Each observation is a **time period** t, represented by features x_t
- SVM chooses (w, b) to define a **decision boundary** $w^\top x + b = 0$ separating Bear from Not-Bear in feature space.
- The decision boundary lies at the **center of a margin**: an empty strip separating the closest observations of the two classes. SVM seeks the **widest possible margin**.
- Unlike logit/probit, SVM directly learns a **classification rule**, not a class probability.
- **Standardize features:** the margin measures distances in feature space, so results depend on the units of measurement.

Why Prefer a Wide Margin?

Two different boundaries can correctly classify the same observations

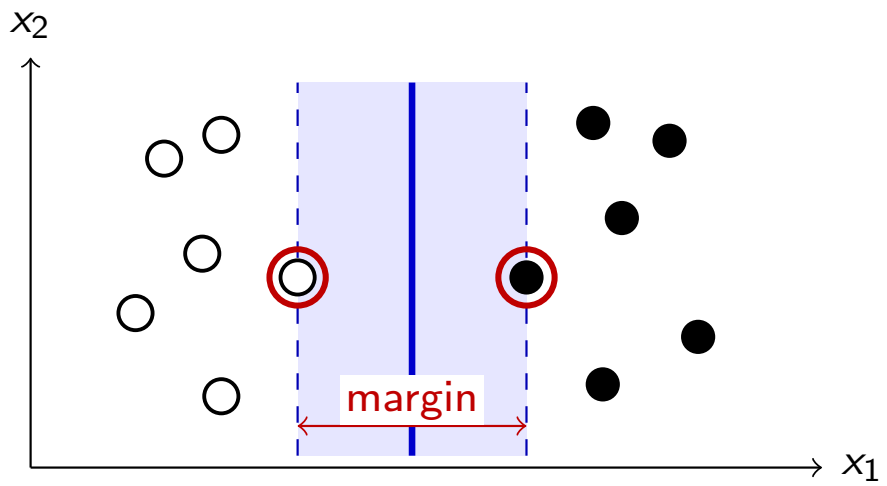


Open = Not-Bear, filled = Bear; solid = decision boundary, dashed = margin edges.

- Both boundaries correctly classify **all training observations**.
- A **wider margin** makes classifications less sensitive to small changes in features, potentially improving **out-of-sample performance**.

What Are Support Vectors?

The closest training observations determine the maximum-margin boundary



- **Support vectors** are the observations touching the two margin boundaries (circled).
- They determine the **location and orientation** of the maximum-margin decision boundary.
- Moving distant points typically has no effect; moving a support vector may change the boundary.

Filled = Bear; open = Not-Bear; circled = support vectors.

Hard-Margin SVM: Geometry

Linearly separable: A straight decision boundary can correctly classify every observation:

$$y_i(w^\top x_i + b) > 0, \qquad i = 1, \ldots, n.$$

Decision boundary: SVM chooses a hyperplane

$$w^\top x + b = 0,$$

where w is the **weight vector**, perpendicular to the decision boundary, and b is the intercept.

Supporting hyperplanes: Normalize (w, b) so the closest observations lie on two parallel hyperplanes:

$$w^\top x + b = -1, \qquad w^\top x + b = +1.$$

Q: What is the **margin** (perpendicular distance between the two supporting hyperplanes) that SVM maximizes?

A:

$$\boxed{\text{Margin} = \frac{2}{\|w\|}}$$

Why Is the Margin $2/\|w\|$?

Example 1: Suppose we have one feature x, with $w = 2$ and $b = 0$. At which feature values are the two supporting boundaries located?

$$2x = -1 \Rightarrow x = -0.5, \qquad 2x = +1 \Rightarrow x = +0.5.$$

$$\text{Margin} = 0.5 - (-0.5) = 1 = \frac{2}{|w|}.$$

Example 2: Consider a different dataset. If $w = 4$ and $b = 0$, where are the supporting boundaries now?

$$4x = -1 \Rightarrow x = -0.25, \qquad 4x = +1 \Rightarrow x = +0.25.$$

$$\text{Margin} = 0.25 - (-0.25) = 0.5 = \frac{2}{|w|}.$$

For a multidimensional x, Margin $= \frac{2}{||w||}$
Larger $\|w\|$ implies a **narrower margin**; smaller $\|w\|$ implies a **wider margin**.

How Do We Maximize the Margin?

Goal: Choose (w, b) to maximize the margin while correctly separating all observations.
We just established that

$$\text{Margin} = \frac{2}{\|w\|}.$$

Thus, **maximizing the margin is equivalent to minimizing** $\|w\|$. We minimize $\frac{1}{2}\|w\|^2$ instead because it is mathematically more convenient:

$$\boxed{\max_{w,b} \frac{2}{\|w\|} \quad \Longleftrightarrow \quad \min_{w,b} \frac{1}{2}\|w\|^2}$$

Subject to correct separation: Each observation must lie on the correct side of its supporting hyperplane:

$$w^\top x_i + b \geq +1 \quad \text{if } y_i = +1,$$
$$w^\top x_i + b \leq -1 \quad \text{if } y_i = -1.$$

Both conditions can be written compactly as

$$y_i(w^\top x_i + b) \geq 1.$$

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- **Noise:** returns are noisy within regimes; feature lead–lag relationships can vary.
- **Overlap:** the distributions of features for bulls and bears often overlap.
- **Label ambiguity:** regime definitions can be arbitrary, so adjacent time periods may have nearly identical features but different labels.

Implication: Rather than require perfect separation, allow some margin violations with a penalty.

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Violations: $0 < \xi_i < 1$ means inside the margin but correctly classified; $\xi_i > 1$ means misclassified.

Tradeoff:

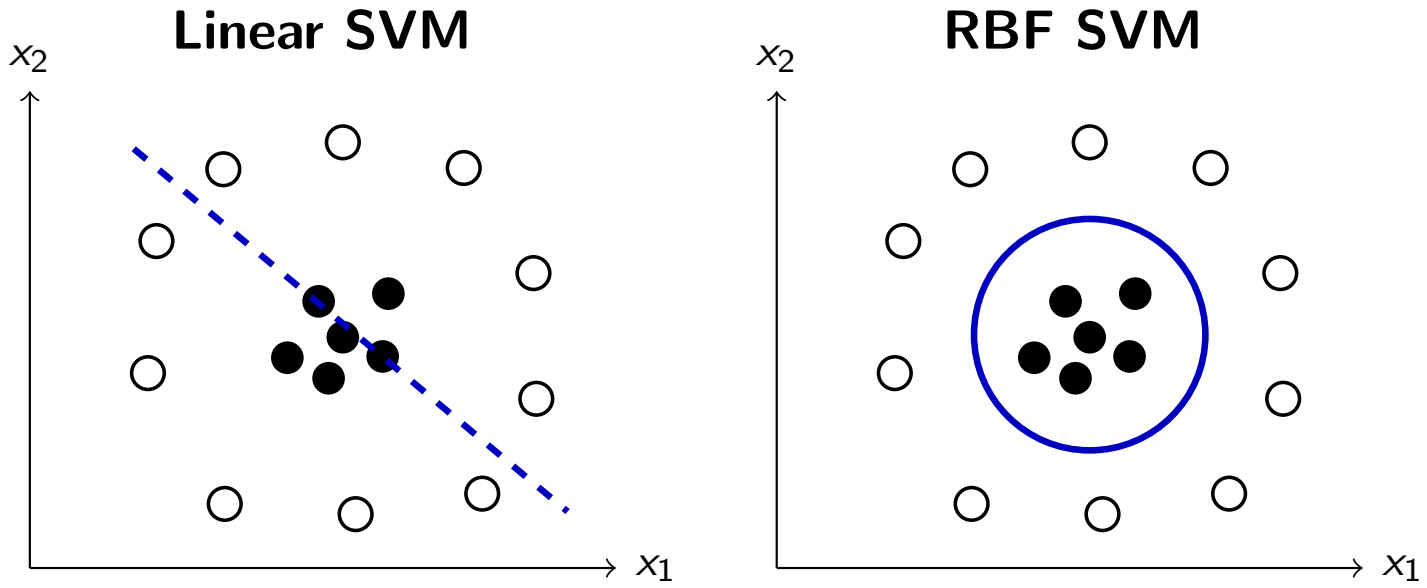
- Larger C: penalize violations more $\Rightarrow$ lower bias, higher variance (less regularization).
- Smaller C: allow violations $\Rightarrow$ more regularization, potentially better generalization.

Loss view: soft-margin SVM is equivalent to regularized empirical risk minimization with hinge loss:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max\{0, 1 - y_i(w^\top x_i + b)\}$$

Why a Linear SVM May Be Insufficient

Problem: Bear and Not-Bear observations need not be separated by a straight line in feature space.



- A **linear SVM** uses $\hat{Y}_t = \text{sign}(w^\top x_t + b)$; a soft margin permits violations, *not curvature*.
- A **kernel SVM** can learn a curved boundary, but additional flexibility may overfit.

Stylized: x_1 = credit spreads, x_2 = volatility; filled = Bear, open = Not-Bear.

Nonlinear SVM

Transform: Fit a linear SVM using nonlinear features $\phi(x)$:

$$f(x_t) = \langle w, \phi(x_t) \rangle + b.$$

Why dot products? At the optimum, SVM weights are sums of transformed features:

$$w = \sum_{s=1}^{n} \alpha_s y_s \phi(x_s), \qquad 0 \leq \alpha_s \leq C.$$

C controls the soft-margin penalty and bounds the dual coefficients α_s.

Substitute: The decision score now depends only on dot products:

$$f(x_t) = \sum_{s=1}^{n} \alpha_s y_s \underbrace{\langle \phi(x_s), \phi(x_t) \rangle}_{\text{dot product}} + b.$$

Implication: To classify x_t, we need the dot products of its transformed features with those of the training observations.

Question: Can we compute these dot products without constructing $\phi(x)$?

The Kernel Trick: A Concrete Example

Suppose we know the transformation for $x = (x_1, x_2)^\top$:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

The dot product we need can be simplified:

$$\begin{aligned} \phi(x_s)^\top \phi(x_t) &= x_{s1}^2 x_{t1}^2 + 2 x_{s1} x_{s2} x_{t1} x_{t2} + x_{s2}^2 x_{t2}^2 \\ &= \boxed{(x_s^\top x_t)^2} \equiv K(x_s, x_t). \end{aligned}$$

This is a kernel: Calculate the *same* dot product directly from the original features, without constructing ϕ.

What if ϕ is much more complicated? Choose a valid kernel that implicitly defines a feature space. For RBF:

$$\boxed{K_\gamma(x_s, x_t) = \exp\left(-\gamma \|x_s - x_t\|^2\right)}.$$

Why exponential? $K = 1$ for identical points and decreases smoothly with distance. RBF is an *exact* inner product in its own infinite-dimensional feature space, not an approximation to an unknown ϕ.

RBF SVM

Optimization: Soft-margin SVM in the transformed feature space:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{s=1}^{n}\xi_s \\ \text{s.t.} \quad & y_s\big(\langle w, \phi(x_s)\rangle + b\big) \geq 1 - \xi_s, \\ & \xi_s \geq 0. \end{aligned}$$

Kernel trick: Instead of constructing $\phi(x)$, use

$$K_\gamma(x_s, x_t) = \langle \phi(x_s), \phi(x_t)\rangle = \exp\left(-\gamma\|x_s - x_t\|^2\right).$$

Resulting classification rule:

$$\hat{Y}_t = \text{sign}\left(\sum_{s\in SV} \alpha_s y_s K_\gamma(x_s, x_t) + b\right).$$

Hyperparameters:

- C: penalty for margin violations; larger C permits less regularization.
- γ: controls the locality of similarity; larger γ permits more flexible boundaries.
- Scale features and choose (C, γ) using CV.

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART – Classification and Regression Trees
4. Evaluating Classification Models

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Let $Y_t \in \{1, 2, 3\}$ denote (bull, neutral, bear), with features X_t.

We can extend probabilistic and margin-based methods to multi-class settings:

- Multinomial logit (softmax probabilities)
- One-vs-rest or one-vs-one SVM

In this subsection we introduce two additional techniques that are naturally suited to multi-class classification: K-NN and Classification Trees. Both methods can also be applied in binary settings.

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART – Classification and Regression Trees
4. Evaluating Classification Models

What is the Probability of Each Market Regime? (k-NN)

Let $Y_t \in \{1, 2, 3\}$ denote (Bull, Neutral, Bear), with $x_t \in \mathbb{R}^K$.
Our goal is now to estimate **three conditional probabilities**:

$$p_{c,t} = P(Y_t = c \mid x_t), \qquad c = 1, 2, 3, \qquad \sum_{c=1}^{3} p_{c,t} = 1.$$

Key idea (k-NN):
Instead of estimating a global model, estimate each class probability locally using the fraction of nearby historical observations in that class.

- Training data: $\{(x_i, y_i)\}_{i=1}^{N}$
- No model coefficients are estimated; predictions use the stored training observations
- The dataset itself serves as the model

Interpretation: If nearby historical feature states were often Bear, the estimated probability of Bear is high relative to the other regimes.

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N).$$

To estimate the probabilities of Bull, Neutral, and Bear at x_t:

1. Compute the distance $d(x_t, x_i)$ from x_t to each historical observation.
2. Identify the k closest observations $\mathcal{N}_k(x_t)$.
3. Estimate each class probability using the neighbors' labels

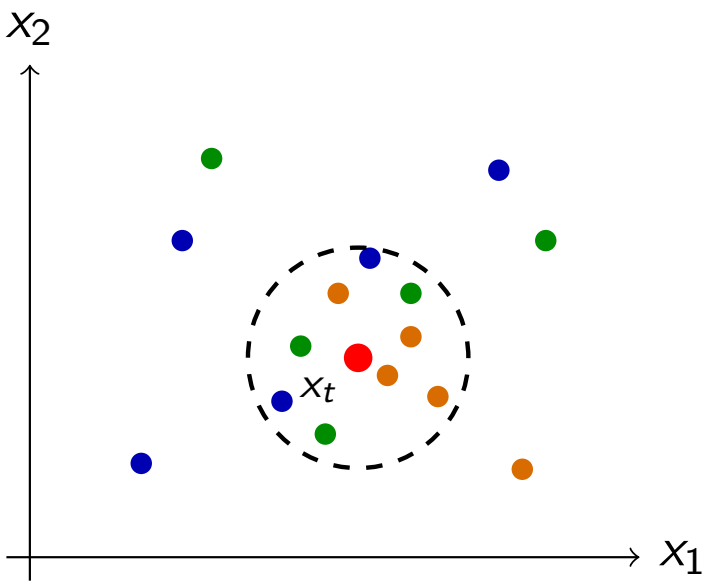
$$\hat{p}_c(x_t) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_t)} \mathbf{1}\{y_i = c\}, \qquad c = 1, 2, 3.$$

Classification rule: $\hat{Y}_t = \arg \max_{c \in \{1,2,3\}} \hat{p}_c(x_t)$.

Distance: Typically Euclidean distance on standardized features. Otherwise, a feature with larger numerical units (e.g., VIX versus returns) may dominate the distance.

k-NN: Geometric Intuition

Each point is a historical observation (x_i, y_i); the red point x_t is the feature vector to classify.



The $k = 9$ nearest neighbors:

- **Bull**: 2 observations
- **Neutral**: 3 observations
- **Bear**: 4 observations

Estimated probabilities:

$$\hat{p}_1(x_t) = 2/9 \approx 0.22,$$
$$\hat{p}_2(x_t) = 3/9 \approx 0.33,$$
$$\hat{p}_3(x_t) = 4/9 \approx 0.44.$$

Prediction: Bear has the largest estimated probability $(4/9)$, so we classify as Bease, even though that probability is below 0.5.

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

- Uses only very similar observations
- Flexible decision boundary
- Low bias, high variance

Large k

- Uses many observations
- Smoother decision boundary
- Higher bias, lower variance

In financial applications, choose k using **time-series cross-validation** (e.g., rolling or expanding training windows). Estimate feature standardization using only the training data in each fold.

Limitation (curse of dimensionality): With many features, distances become less informative, especially when some features are irrelevant.

K-means vs k-NN

K-means	**k-NN**
Unsupervised (uses only X)	Supervised (uses (X, Y))
Learns K cluster centroids	No parametric training (stores data)
Assigns points to nearest centroid	Predicts label via nearest neighbors' labels
Output: cluster ID	Output: class label / class probabilities

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART – Classification and Regression Trees
4. Evaluating Classification Models

What is the Probability of Each Market Regime?

Let $Y_t \in \{1, 2, 3\}$ denote (Bull, Neutral, Bear), with $x_t \in \mathbb{R}^K$.
Our goal is to estimate

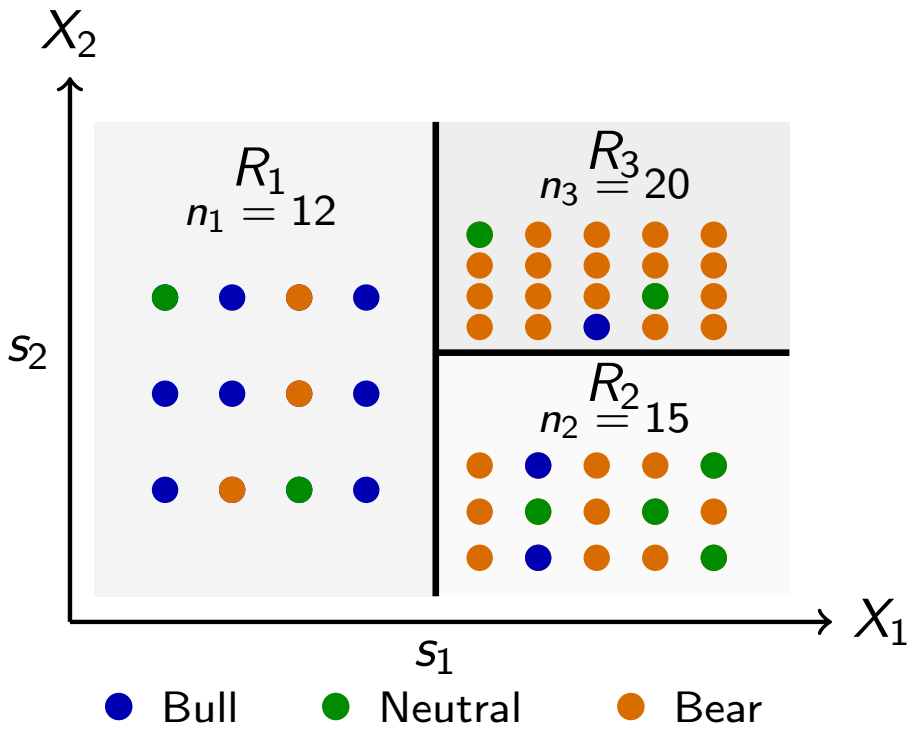
$$p_{c,t} = P(Y_t = c \mid x_t), \qquad c = 1, 2, 3.$$

Key idea (CART): Partition the feature space into regions using a sequence of simple decision rules.
For an observation assigned to a terminal region (*leaf*) R_m, estimate each class probability using the observed frequency in that leaf:

$$\hat{p}_{c,m} = \frac{\#\{\text{class } c \text{ observations in } R_m\}}{\#\{\text{observations in } R_m\}}, \qquad c = 1, 2, 3.$$

All observations in the same leaf receive the same *three* estimated class probabilities, which sum to one.

Splitting the Feature Space



- Each dot is a historical observation; regions can contain different numbers of dots.
- Each split uses one feature and one threshold.

Estimated class probabilities:

$$\hat{\boldsymbol{p}}_1 = (7, 2, 3)/12,$$

$$\hat{\boldsymbol{p}}_2 = (2, 4, 9)/15,$$

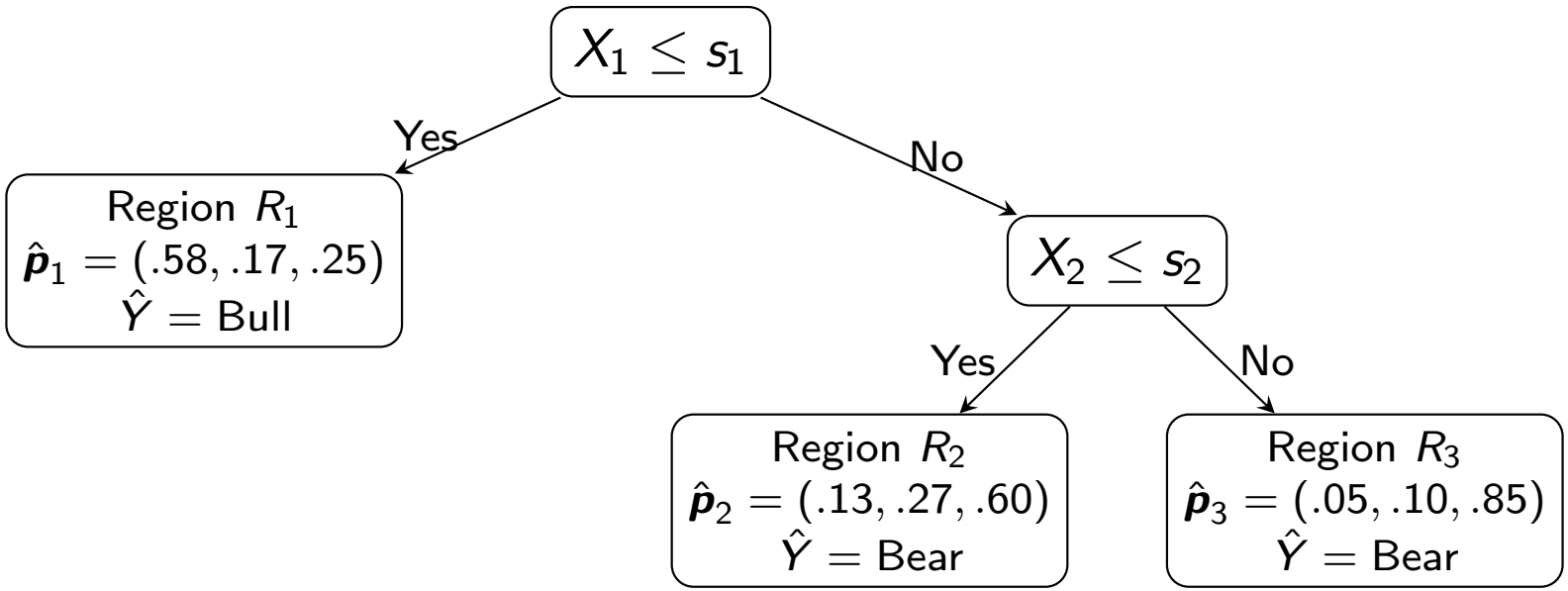
$$\hat{\boldsymbol{p}}_3 = (1, 2, 17)/20.$$

Each vector is ordered (Bull, Neutral, Bear).

Interpretation: The split on X_2 applies only when $X_1 > s_1$. Thus CART can capture nonlinearities and interactions through successive splits.

Equivalent Tree Representation

The three regions in the previous figure can be represented by a decision tree.



Each $\hat{\boldsymbol{p}}_m$ is the estimated (Bull, Neutral, Bear) probability vector.
A new observation follows the decision rules to a leaf; its predicted class is the one with the largest leaf probability.

How CART Grows a Classification Tree

At each node, CART:

1. For each feature j, sort its observed values and consider thresholds between consecutive distinct values (e.g., their midpoints):

$$\{i : x_{ij} \leq s\} \qquad \text{and} \qquad \{i : x_{ij} > s\}.$$

2. Calculates the reduction in classification impurity (e.g., Gini) for each candidate split.
3. Selects the feature and threshold (j, s) that produce the **largest impurity reduction**.
4. Repeats the process in each child node until a stopping criterion is reached (e.g., maximum depth or minimum leaf size).

Greedy optimization: Each split is best at that node; CART does not search for the globally best sequence of splits.

Impurity for Classification: Gini

For a node A with empirical class proportions $\{\hat{p}_{c,A}\}_{c=1}^{C}$:

$$G(A) = 1 - \sum_{c=1}^{C} \hat{p}_{c,A}^2.$$

For our **three regimes** (Bull, Neutral, Bear):

$$G(A) = 1 - \left(\hat{p}_{1,A}^2 + \hat{p}_{2,A}^2 + \hat{p}_{3,A}^2\right).$$

- $G(A) = 0$: the node is pure (all observations belong to one class).
- $G(A) = 2/3$: the node is maximally mixed (one-third in each class).

For a parent P and children L, R, evaluate the **reduction in weighted Gini impurity**:

$$\Delta G = G(P) - \left[\frac{n_L}{n_P} G(L) + \frac{n_R}{n_P} G(R)\right].$$

CART prefers the candidate split with the largest ΔG.

Why Does Gini Measure Impurity?

Suppose a node has three class proportions:

$$\hat{p}_{\text{Bull}} = 0.50, \qquad \hat{p}_{\text{Neutral}} = 0.30, \qquad \hat{p}_{\text{Bear}} = 0.20.$$

Thought experiment: Randomly predict a class using these same proportions, independently of the observation's actual class.

Multiplication rule (independent events):

$$P(\text{Actual } c \textbf{ and } \text{Predict } c) = \hat{p}_c \times \hat{p}_c = \hat{p}_c^2.$$

Addition rule (mutually exclusive events): A correct prediction is Bull/Bull *or* Neutral/Neutral *or* Bear/Bear.

$$P(\text{Correct}) = 0.50^2 + 0.30^2 + 0.20^2 = 0.38.$$

Thus the probability of an incorrect prediction is

$$\boxed{G(A) = P(\text{Incorrect}) = 1 - P(\text{Correct}) = 0.62.}$$

Interpretation: Gini measures class mixing using a hypothetical random prediction. CART actually predicts the most likely class.

Example: Gini Reduction from a Split

After splitting on X_1, consider the 35 observations with $X_1 > s_1$; a candidate split on $X_2 \leq s_2$ gives:

	Bull	Neutral	Bear	Total
Parent ($R_2 \cup R_3$)	3	6	26	35
$X_2 \leq s_2$ (R_2)	2	4	9	15
$X_2 > s_2$ (R_3)	1	2	17	20

$$G(P) = 1 - \left[\left(\frac{3}{35}\right)^2 + \left(\frac{6}{35}\right)^2 + \left(\frac{26}{35}\right)^2\right] \approx 0.411.$$

$$G(R_2) = 1 - \frac{2^2 + 4^2 + 9^2}{15^2} \approx 0.551, \quad G(R_3) = 1 - \frac{1^2 + 2^2 + 17^2}{20^2} = 0.265.$$

$$\Delta G \approx 0.411 - \left[\frac{15}{35}(0.551) + \frac{20}{35}(0.265)\right] \approx \boxed{0.024}.$$

Interpretation: Even though R_2 is less pure than its parent, the *weighted* impurity falls. The most likely class in both R_2 and R_3 is Bear.

Controlling Overfitting: Pre-pruning

Very deep trees can fit training noise and produce unreliable probabilities

Suppose a leaf contains just one observation, which happens to be Bear:

$$(\hat{p}_{\text{Bull}}, \hat{p}_{\text{Neutral}}, \hat{p}_{\text{Bear}}) = (0, 0, 1).$$

That leaf is perfectly pure in the training data, but one observation provides little evidence that the true probability of Bear is 100%.

Pre-pruning (early stopping): Limit tree growth using hyperparameters such as

- maximum tree depth;
- minimum number of observations per leaf;
- minimum impurity reduction required to split.

Select complexity settings using validation data or cross-validation, rather than training fit alone.

Controlling Overfitting: Post-pruning

Grow a relatively large tree, then remove branches that add little value

Gini-based cost-complexity pruning: Choose a subtree T of the initially grown tree T_0 that balances impurity against complexity:

$$\hat{T}_\alpha = \arg\min_{T \preceq T_0} \left\{ \widehat{R}_G(T) + \alpha \, |\mathcal{L}(T)| \right\}.$$

Here $\mathcal{L}(T)$ is the set of terminal leaves, and the tree's weighted leaf impurity is

$$\widehat{R}_G(T) = \sum_{m \in \mathcal{L}(T)} \frac{N_m}{N} \underbrace{\left(1 - \sum_{c=1}^{3} \hat{p}_{c,m}^2 \right)}_{G(R_m)}.$$

- $\widehat{R}_G(T)$ measures the remaining training impurity.
- $|\mathcal{L}(T)|$ counts the leaves (tree complexity).
- Larger α favors a smaller tree; choose α by validation or cross-validation.

CART for Regression

The same recursive partitioning procedure applies when the outcome is continuous, such as an asset return $Z_t \in \mathbb{R}$.

Leaf prediction: Replace the Bear frequency with the average outcome in region R_m:

$$\hat{z}_t = \bar{z}_m = \frac{1}{N_m} \sum_{i:x_i \in R_m} z_i.$$

Splitting criterion: Instead of reducing Gini impurity, choose splits that reduce the sum of squared errors within the child nodes:

$$\mathrm{SSE}(L) + \mathrm{SSE}(R) = \sum_{i \in L} (z_i - \bar{z}_L)^2 + \sum_{i \in R} (z_i - \bar{z}_R)^2.$$

- **Classification tree:** predict class probabilities; typically split using Gini impurity.
- **Regression tree:** predict a numeric value; typically split using squared error.

Both produce piecewise-constant predictions over regions of the feature space.

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
4. Evaluating Classification Models

How Do We Evaluate a Classifier?

Different models may produce a predicted class $\widehat{Y}_{t+1}$, a probability $\hat{p}_{t+1}$, or a vector of probabilities.

We want to evaluate several different dimensions of model performance:

- **Classification:** how often do we correctly identify the regime?
- **Ranking:** does the model assign higher scores to Bear periods than Not-Bear periods?
- **Probabilities:** how good are the estimated probabilities?
- **Economic decisions:** how costly are the model's mistakes?

These questions apply to both binary and multi-class classification.

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$ (Among all Bear periods, the fraction correctly identified)
- $Specificity = \frac{TN}{TN+FP}$ (Among Not-Bear periods, the fraction correctly identified)
- Type I Error $= \frac{FP}{FP+TN}$ (False alarm rate aka False positive rate: predicting Bear when it was Not-Bear)
- Type II Error $= \frac{FN}{TP+FN}$ (Missed crisis rate: failing to predict Bear)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).
Q: Suppose we have 100 observations. Can you fill in the confusion matrix?

- $TP = 0$ (never correctly predicts Bear)
- $FN = 10$ (misses every Bear)
- $FP = 0$ (never raises a false alarm)
- $TN = 90$ (always correctly predicts Not-Bear)

Q: What is accuracy in this model?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{90}{0 + 10 + 0 + 90} = 90\%$$

Conclusion: with rare Bear markets, accuracy can be dominated by TN and can reward useless models.

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** do we catch crises?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score (harmonic mean):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Penalizes models that have high precision but low recall (miss crises), or high recall but low precision (constant false alarms).
- Helpful in rare-event settings because it emphasizes performance on the Bear class (it does not "get credit" for many easy true negatives).

Multi-Class Confusion Matrix

Bull / Neutral / Bear

For multi-class classification, the confusion matrix expands from 2×2 to $K \times K$.

Suppose we observe 100 periods:

	Predict Bull	Predict Neutral	Predict Bear
Actual Bull	60	8	2
Actual Neutral	6	12	2
Actual Bear	4	4	2

Q: What is accuracy?

$$Accuracy = \frac{60 + 12 + 2}{100} = 74\%$$

Q: What is recall for the Bear class?

$$Recall_{Bear} = \frac{2}{4 + 4 + 2} = 20\%$$

Conclusion: the model has 74% accuracy, but misses 80% of Bear periods.

Evaluating Multi-Class Classification

Macro F1 and Balanced Accuracy

Treat each class k as the positive class, with all other classes grouped together. For each class, compute:

$$Precision_k = \frac{TP_k}{TP_k + FP_k}, \qquad Recall_k = \frac{TP_k}{TP_k + FN_k}$$

$$F1_k = 2 \cdot \frac{Precision_k \cdot Recall_k}{Precision_k + Recall_k}$$

Macro F1: average F1 across all K classes.

$$\text{Macro F1} = \frac{1}{K} \sum_{k=1}^{K} F1_k$$

Balanced Accuracy: average recall across all K classes.

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{k=1}^{K} Recall_k$$

Both give equal weight to Bull, Neutral, and Bear, regardless of how frequent each class is.

Evaluating Multi-Class Classification (cont'd)

Recall our earlier confusion matrix (70 Bull, 20 Neutral, 10 Bear):

$$Recall_{Bull} = \frac{60}{70} \approx 86\%, \qquad Recall_{Neutral} = \frac{12}{20} = 60\%$$

$$Recall_{Bear} = \frac{2}{10} = 20\%.$$

$$\text{Balanced Accuracy} = \frac{86\% + 60\% + 20\%}{3} \approx 55\%$$

The same confusion matrix gives Macro F1 $\approx 55\%$, compared with $Accuracy = 74\%$.

Other common averages:

- **Weighted F1:** weight each class by its frequency.
- **Micro F1:** aggregate across classes (equals accuracy for single-label multi-class classification).

Interpretation: macro measures are useful when good performance on rare regimes is important.

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

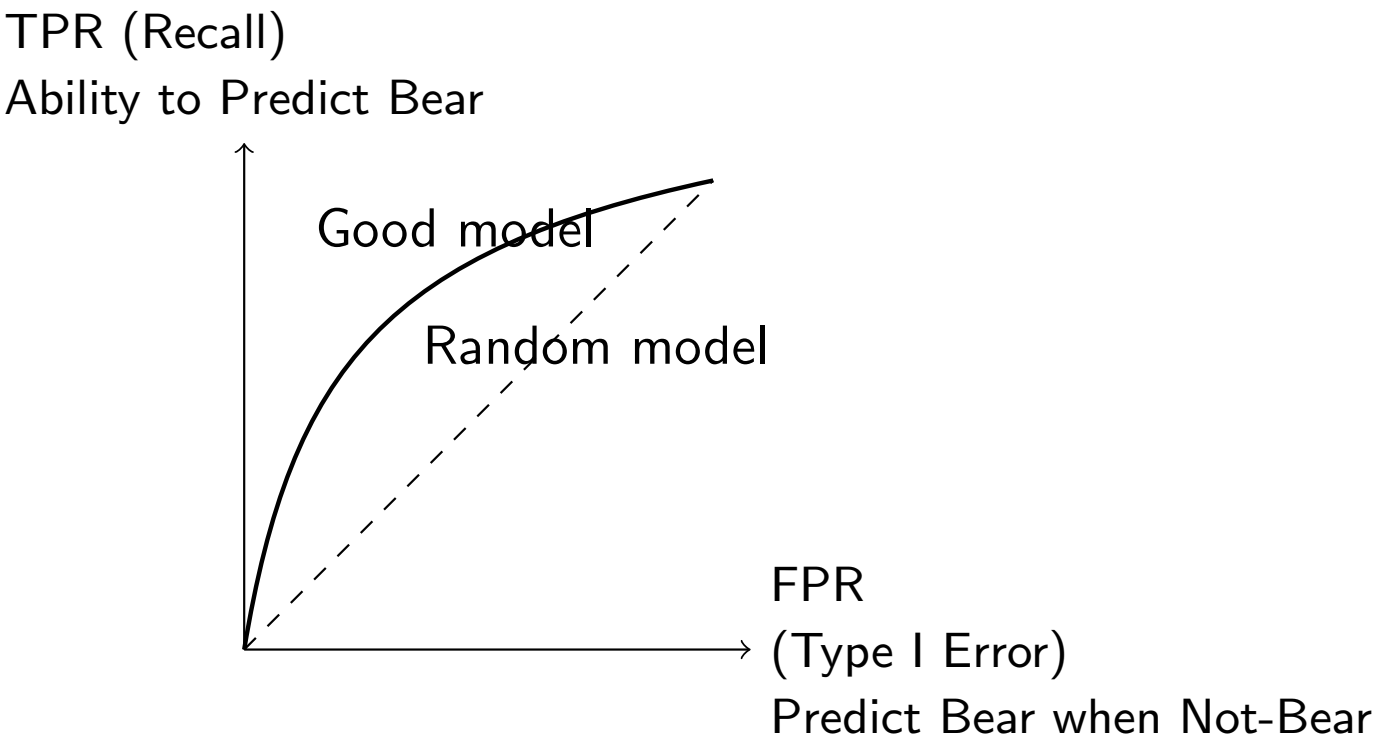
$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

(For distinct scores; the standard definition gives half credit for ties.)

- Probability that a randomly chosen Bear period receives a higher predicted score than a randomly chosen Not-Bear period.
- Evaluates the model as a **ranker**, not a classifier.
- $AUC = 0.5$: random ordering $\qquad$ $AUC = 1$: perfect separation.

Interpretation: Even if the cutoff changes, a high-AUC model consistently places

ROC Curve Illustration



- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to random ordering.
- $AUC = 1$ corresponds to perfect ranking.

Interpretation: A high-AUC model improves crisis detection (vertical axis) without generating excessive false alarms (horizontal axis).

ROC and AUC for Multi-Class Classification

We can extend ROC/AUC to multi-class problems using a **one-vs-rest** approach.

- **Bull vs Rest:** treat Bull as positive and Neutral/Bear as negative.
- **Neutral vs Rest:** treat Neutral as positive and Bull/Bear as negative.
- **Bear vs Rest:** treat Bear as positive and Bull/Neutral as negative.

For each class, compute its own ROC curve and AUC_k.

$$\text{Macro AUC} = \frac{1}{K}\sum_{k=1}^{K} AUC_k$$

- Measures how well each class is ranked against the other classes.
- ROC/AUC requires a score for each class, but not necessarily a calibrated probability.

Interpretation: a high Bear-vs-rest AUC means Bear periods tend to receive higher Bear scores than other periods.

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss. Define the costs:

- C_{FP} = cost of false alarm (predict Bear when Not-Bear)
- C_{FN} = cost of missed crisis (predict Not-Bear when Bear)

Let the model estimate

$$p(X) = P(Y = 1 \mid X)$$

Expected loss conditional on X:

$$L(\text{Bear} \mid X) = C_{FP}(1 - p(X))$$

$$L(\text{Not-Bear} \mid X) = C_{FN}p(X)$$

Choose Bear when

$$C_{FP}(1 - p(X)) < C_{FN}p(X)$$

How Do We Choose the Threshold τ? cont'd

Which implies the optimal threshold

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- If missed crises are very costly $\Rightarrow$ choose low τ.
- If false alarms are very costly $\Rightarrow$ choose high τ.

Important: the costs C_{FP} and C_{FN} reflect user preferences and are chosen based on the economic context.

Economic Costs in Multi-Class Classification

With Bull, Neutral, and Bear, we may face different costs for different mistakes.
Let $C(a, k)$ be the cost of choosing action a when the actual class is k.
If the model estimates $\hat{p}_k(X) = P(Y = k \mid X)$, then

$$L(a \mid X) = \sum_{k=1}^{K} C(a, k)\hat{p}_k(X)$$

Choose the action with the lowest expected loss:

$$\widehat{Y}(X) = \arg\min_{a} \sum_{k=1}^{K} C(a, k)\hat{p}_k(X)$$

Interpretation: predicting Bull when the market is Bear may be much more costly than predicting Neutral when the market is Bear.
The class with the highest estimated probability need not be the economically optimal decision.

Multi-Class Decision Example

Choosing the Lowest Expected Loss

Suppose the model predicts

$$\hat{p}_{Bull} = 0.55, \qquad \hat{p}_{Neutral} = 0.30, \qquad \hat{p}_{Bear} = 0.15.$$

Assume the following illustrative costs:

Decision \ Actual	Bull	Neutral	Bear
Predict Bull	0	2	10
Predict Neutral	1	0	4
Predict Bear	3	2	0

Expected loss for each decision:

$$L(Bull \mid X) = 0(0.55) + 2(0.30) + 10(0.15) = 2.10$$
$$L(Neutral \mid X) = 1(0.55) + 0(0.30) + 4(0.15) = 1.15$$
$$L(Bear \mid X) = 3(0.55) + 2(0.30) + 0(0.15) = 2.25$$

Conclusion: choose Neutral, even though Bull has the highest probability.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.
- Penalizes overconfident wrong forecasts.

Interpretation examples:

- Predict 0.9 probability of Bear and no Bear occurs $\Rightarrow$ large penalty.
- Predict 0.6 probability of Bear and Bear occurs $\Rightarrow$ small penalty.

Interpretation: Good probability predictions matter. Overconfident crisis forecasts can be economically costly.

Multi-Class Brier Score

Evaluating the Entire Probability Vector

For Bull, Neutral, and Bear, the model estimates $\hat{p}_{t+1,k}$ for each class k. The multi-class Brier score is

$$\text{Brier Score} = \frac{1}{2T} \sum_{t=1}^{T} \sum_{k=1}^{K} \left(\mathbf{1}\{Y_{t+1} = k\} - \hat{p}_{t+1,k} \right)^2$$

(The factor $1/2$ makes this equal to the earlier binary Brier score when $K = 2$.)

Example: suppose the model predicts

$$(\hat{p}_{Bull}, \hat{p}_{Neutral}, \hat{p}_{Bear}) = (0.6, 0.3, 0.1)$$

and a Bear market actually occurs. Then the contribution is

$$\frac{1}{2} \left[(0 - 0.6)^2 + (0 - 0.3)^2 + (1 - 0.1)^2 \right] = 0.63.$$

Interpretation: the model receives a large penalty for assigning only 10% probability to the Bear outcome that actually occurred.

Log Loss

Another Way to Evaluate Probability Predictions

For multi-class outcomes, let $\hat{p}_{t+1,Y_{t+1}}$ be the probability assigned to the class that actually occurs.

Log Loss (Cross-Entropy):

$$\text{Log Loss} = -\frac{1}{T}\sum_{t=1}^{T} \log \hat{p}_{t+1,Y_{t+1}}$$

- Lower values indicate better probability predictions.
- Penalizes models that assign very low probability to the outcome that actually occurs.
- Like the Brier score, it is a proper scoring rule.

Interpretation example: suppose a Bear market occurs.

- Predict $\hat{p}_{Bear} = 0.10 \Rightarrow \text{loss} = -\log(0.10) \approx 2.30$.
- Predict $\hat{p}_{Bear} = 0.60 \Rightarrow \text{loss} = -\log(0.60) \approx 0.51$.

Important: AUC can use scores, but Brier and log loss require valid probability estimates (e.g., an SVM's raw score is not a probability).

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)

3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART – Classification and Regression Trees

4. Evaluating Classification Models